Scalable Technologies for Data Analysis

Quality, Tests, Observability and Costs

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

Quality, Tests, Observability and Costs

From correct pipelines to trustworthy systems

- By the end of this class you should be able to
  - Explain why correctness alone is not enough for production data systems, and identify the additional requirements (quality, observability, cost)
  - Define and apply data quality dimensions (e.g. correctness, completeness, freshness, consistency) across different layers of a data platform
  - Design validation strategies and quality gates within pipelines, ensuring safe and reliable data publication
  - Distinguish between different types of tests in data systems (unit, integration, contract) and apply them appropriately
  - Understand and implement observability principles for data pipelines, including logs, metrics, and tracing
  - Diagnose common data pipeline issues using observable signals (e.g., volume changes, failures, performance regressions)
  - Explain how data incidents occur, classify their impact, and outline appropriate response strategies
  - Identify the main drivers of performance and cost in distributed data processing, and reason about trade-offs
  - Apply evidence-based optimization and cost-aware design decisions in data pipelines and serving layers

The plan

1. Intro to modern Data Platforms and scalability
2. Contract-Driven Design and Data Flow Modeling
3. Storage + table formats
4. Processing patterns + optimization
5. Orchestration + production reliability
6. Serving: SQL/APIs/semantic layer
7. Quality + observability + cost
8. Incremental + streaming
9. ML integration
10. Governance + evolution

Previously…

- In the previous classes, we
  - Designed data platforms around data products, contracts, and layers (bronze/silver/gold)
  - Learned how to store and organize data efficiently (Parquet, partitioning, Iceberg)
  - Built scalable transformations, understanding cost drivers (scan, shuffle, skew)
  - Implemented reliable pipelines using orchestration (Flyte), with retries, idempotency, and backfills
  - Designed serving layers (SQL, materialization, APIs) with clear contracts and trade-offs
- Today
  - We don't add a new layer, we discuss horizontal concerns
  - We move from building pipelines to operating data systems in production
  - We focus on what makes systems trustworthy and sustainable at scale

Introduction

Data quality is not (only) a “cleanup step”

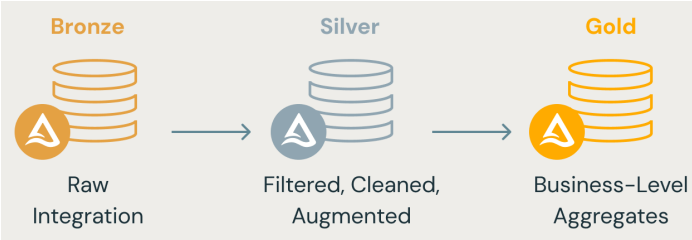
- Data quality is often misunderstood as
  - Fixing nulls
  - Removing duplicates
  - Correcting formats
- But in production systems, data quality is broader: it defines whether a dataset can be trusted and safely consumed
- A dataset may be
  - Syntactically valid
  - Successfully processed
  - Stored in the right place
- …and still be wrong for business use
- Examples
  - Revenue is complete, but semantically incorrect
  - A table is fresh, but contains duplicated orders
  - A dashboard loads, but uses inconsistent definitions across teams
- The key idea is that quality is not only about data values, it is about fitness for use, relative to contracts, consumers, and decisions

Data quality dimensions

- Data quality has multiple dimensions, not just one notion of correctness
- Common dimensions include
  - Correctness: values reflect reality or the intended business logic
  - Completeness: required records or fields are present
  - Consistency: the same concept is represented the same way across datasets and time
  - Uniqueness: the same entity/event is not unintentionally duplicated
  - Validity: values respect type, format, range, and domain rules
  - Freshness: data is available within the expected time window
- One important challenge to have in mind is that quality dimensions may conflict
  - e.g. maximizing freshness may reduce validation depth
- Therefore, quality must be designed as a set of explicit expectations, not as an informal feeling that "the data looks fine"

Quality depends on the layer

- Data quality expectations are not the same in Bronze, Silver, and Gold
- The key idea to bear in mind is that quality becomes stricter as data moves closer to consumers



Bronze (raw/traceable)

- Main goal: preserve fidelity
- Typical checks
  - Schema can be parsed
  - Required ingestion metadata exists
  - Source and batch provenance are recorded
- Tolerance: nulls, duplicates, malformed values may still exist
- The objective is auditability, not business readiness

Silver (clean / conformed)

- Main goal: produce reliable, reusable data
- Typical checks
  - Deduplication rules
  - Type normalization
  - Nullability constraints
  - Key validity
  - Timestamp/timezone standardization
- The objective is consistency and trust for downstream reuse

Gold (business / serving)

- Main goal: publish decision-ready outputs
- Typical checks
  - Metric semantics
  - Aggregation correctness
  - Business rule validation
  - Freshness / availability SLOs
- The objective is stable consumption and decision support

Quality must be tied to contracts

- A data contract defines the expected interface of a dataset
  - Schema
  - Grain
  - Keys
  - Semantics
  - Constraints
  - SLOs
- Therefore, quality is not arbitrary, it should be derived from the contract
- Examples
  - If the contract states one row per order, uniqueness becomes testable
  - If the contract defines net revenue semantics, business validation becomes possible
  - If the contract defines freshness expectations, lateness becomes measurable
- This changes the role of quality checks from ad-hoc “sanity checks” to contract-aware validation
- Good quality checks verify promises they do not just inspect data mechanically

Validation gates: when do we stop the pipeline?

- A validation gate is a decision point before data is published or promoted. Its purpose is to answer *"Is this output safe to expose to downstream consumers?"*
- Typical validation gate examples:
  - Row count is within expected range
  - No duplicated business keys
  - Freshness target is satisfied
  - Metric totals are not implausible
  - Schema is compatible with the contract
- Possible outcomes
  - Fail and block publication
  - Publish with warning
  - Publish to a lower-trust layer only
- Practical principle
  - The closer the data is to Gold, the less tolerance we should have
- Quality gates connect naturally to aspects we have already covered in previous classes
  - Deterministic pipelines
  - Idempotent reruns
  - Safe publishing patterns in orchestration

Testing in data pipelines

Why testing data pipelines is different

- In traditional software
  - Inputs are controlled
  - Outputs are deterministic and small
  - Behavior is easier to isolate
- In data systems
  - Inputs are large, evolving, and often messy
  - Outputs depend on time, upstream systems, and data quality
  - Failures may not crash the system: they may produce wrong results
- Common problems
  - Joins that duplicate data (row explosion)
  - Silent schema changes
  - Incorrect aggregations due to grain mismatch
  - Late or missing data affecting metrics
- Key challenge: data systems fail silently more often than they fail loudly
- Therefore testing is not optional: it is essential to prevent undetected errors

Types of tests in data systems

- We can adapt classic testing concepts to data pipelines
- Unit tests: test individual transformations or functions. Compare small, controlled inputs vs. expected outputs
  - Aggregation logic
  - Deduplication rules
  - Timestamp normalization
- Integration tests: test multiple steps together (pipeline segments or full DAGs)
  - Bronze > Silver > Gold flow
  - End-to-end metric computation
- Contract tests: validate that data respects its contract
  - Schema matches expected structure
  - Primary keys are unique
  - Grain is respected (no unintended duplication)
  - Required fields are not null
- Key idea: different tests catch different failure modes

Testing vs. runtime validation

- It is important to distinguish between what is testing and what is runtime validation
- Testing (pre-deployment)
  - Happens before code reaches production
  - Validates logic and assumptions
  - The goal is to prevent introducing bugs
- Validation (runtime checks)
  - Happens during pipeline execution
  - Uses real data
  - The goal is to detect issues caused by upstream changes, data anomalies or unexpected distributions
- Examples
  - Test: “dedup logic works for edge cases”
  - Validation: “today’s data has no duplicate keys”
- Key idea
  - Tests protect logic
  - Validation protects execution

The role of synthetic and edge-case data

- Real production data is large, noisy, hard to control
- Whereas tests need small, controlled datasets
- Synthetic data allows us to
  - Simulate edge cases
  - Reproduce tricky scenarios
  - Ensure deterministic behavior
  - Have an evaluation benchmark for automated evaluation of data pipelines
- Important edge cases
  - Null values in critical fields
  - Duplicated keys/events
  - Out-of-order timestamps
  - Extreme values (e.g. very large / small amounts)
  - Skewed distributions
- Without synthetic data, many bugs only appear in production (and usually at the worst time)
- Think that if you don't design edge cases, production will do it for you!

Where tests live in a data platform

- In a data platform, such as in out stack, tests are not a single component: they are distributed across different components. They are, for this reason, horizontal, closer to Governance than to an individual element. They are distributed across
  - Transformation code: unit tests for logic (Python/SQL)
  - In pipeline definitions: validation tasks (e.g. before publishing Gold)
  - In contracts: schema and constraint validation
  - In orchestration: gating execution based on test results
- As a good practice, tests should be
  - Automated
  - Versioned
  - Reproducible
- A common anti-pattern is “manual validation in dashboards”
  - If it reached the dashboard, it’s already too late…
- Testing must be part of the system design, not an afterthought

Observability

From detection to action

The “black box” problem

- A pipeline may be running, scheduled or producing outputs, but we don’t know what is actually happening inside
- Common questions often posed in production
  - Which step is slow?
  - Did the data arrive?
  - Why did this metric change?
  - What failed, and where?
- Without observability
  - Failures are hard to detect
  - Debugging is slow and reactive
  - Trust in the system decreases
- This is the “black box” problem: the system runs, but we cannot see or explain its behavior
- And, if you cannot observe a system, you cannot operate it

The three pillars of observability

- Observability is built on three complementary signals: logs, metrics and tracing
- Logs are discrete records of events, such as
  - "Task started"
  - "Read 10M rows"
  - "Validation failed"
- Metrics are aggregated, numerical measurements over time, such as
  - Runtime duration
  - Number of rows processed
  - Error counts
- Tracing tracks how a request or pipeline execution flows across components, including
  - Which tasks ran
  - Dependencies between steps
  - Where time is spent
- Together, they answer
  - What happened? (logs)
  - How much / how often? (metrics)
  - Where and why? (traces)

What should we observe in data pipelines?

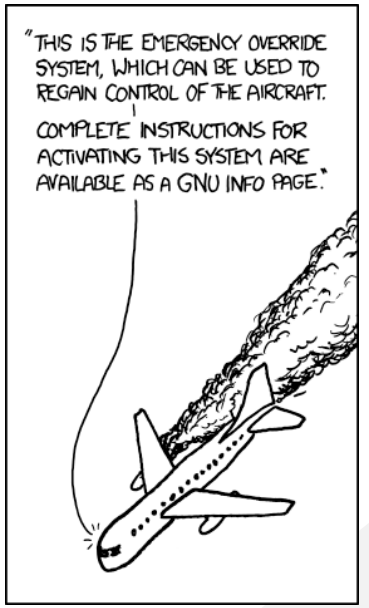
- Observability is not just about infrastructure or adopting a certain technology: it must include data behavior. We must observe both system behavior and data behavior
- Freshness
  - Is data arriving on time?
  - Are SLOs being met?
- Volume
  - Number of rows / bytes processed
  - Sudden drops or spikes
- Data distribution
  - Changes in values (drift) (e.g. country distribution changes unexpectedly)
- Failures and retries
  - Task failures
  - Retry patterns
  - Timeout occurrences
- Performance signals
  - Runtime
  - Input data scanned
  - Shuffle / network usage
  - Memory pressure / spills
- All of these directly connect to cost and performance drivers we have seen earlier

Observability as contract monitoring

- Observability is not only technical: it is also semantic. In every step we should ask: does the data still respect its contract?
- Examples
  - Uniqueness constraint violated
  - Unexpected null rates
  - Metric values outside expected ranges
  - Freshness SLO missed
- This connects observability with
  - Data contracts (what we promised)
  - Quality checks (what we validate)
- When we come from a traditional “Software Engineering background” we must often change the mindset
  - From “is the pipeline running?”
  - To “is the data still correct and usable?”
- Observability should validate system health as well as data correctness

From alerts to runbooks

- Detecting a problem is not enough. A good system must answer: what should we do when this fails?
- Alerts are triggered when something goes wrong and are usually shown on a dashboard, or by sending a message to some channel
  - Pipeline failure
  - Freshness delay
  - Abnormal metric values
- Runbooks, on the other hand, are predefined steps to diagnose and resolve issues
- Example
  - Step 1: check upstream ingestion
  - Step 2: inspect failed task logs
  - Step 3: validate affected partitions
  - Step 4: rerun specific window
- Without runbooks, we have only alerts, which can create noise
  - Resolution also depends on “hero engineers”
- The key message is that observability must lead to actionable responses, that anyone can follow

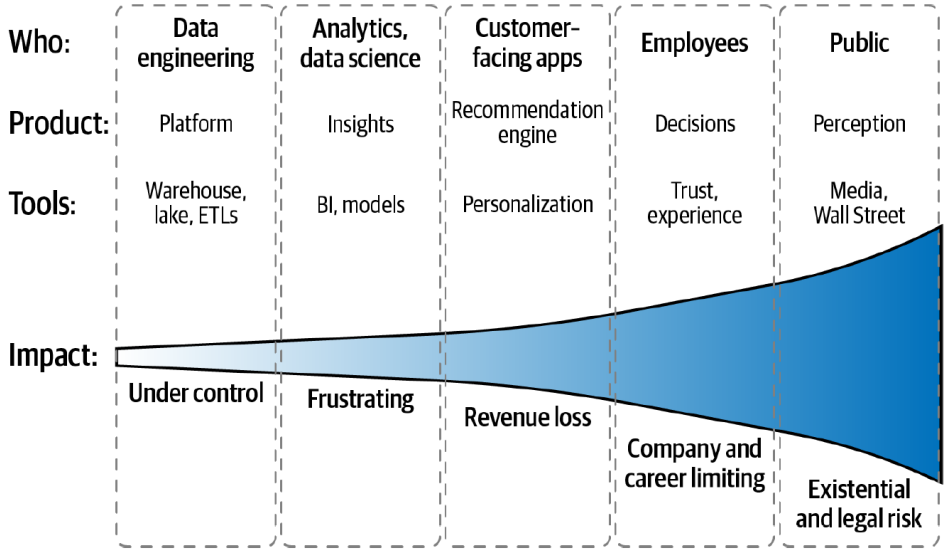


What is a data incident?

- A data incident is any event where data is incorrect, missing, or late, and it impacts downstream consumers
- Important distinction
  - Systems may still be running
  - Pipelines may still succeed
- ...but the data is wrong
- Examples
  - KPI suddenly drops due to upstream schema change
  - Duplicate records inflate metrics
  - Daily table not updated before business hours
  - API serving stale data
- In data systems, as opposed to more traditional software engineering environments, “no error” does not necessarily mean “no problem”

Data downtime and impact

- Data downtime are periods where data is unavailable, incorrect, or unreliable
- Also, not all incidents are equal: we need severity levels
- High
  - Business-critical dashboards are wrong
  - Financial or operational decisions affected
- Medium
  - Partial data missing or delayed
  - Some consumers impacted
- Low
  - Minor inconsistencies
  - Internal or non-critical datasets affected



- Why severity matters
  - Prioritization of response, especially when there are multiple incidents and/or limited resources
  - Communication urgency (i.e. should we wake someone up?)
  - Allocation of resources
- Incidents must be managed not only by cause, but also by impact

Incident response lifecycle

- When incidents occur, the organization should have a structured response. This usually reduces chaos, recovery time and dependence on specialized individuals (heroes)

1. Detection, by looking at observability signals such as alerts, anomalies, failures
2. Diagnosis, in which we identify where the issue occurred, what changed, which data is affected
3. Mitigation, through short-term actions including stop downstream usage, rollback / revert, isolate affected partitions
4. Resolution, by fixing the root cause, which can reside in code, schema, logic or be an upstream issue
5. Communication, through which we inform stakeholders, by letting them know about what happened, what was the impact, and what is the expected resolution

- The key idea is that Incident handling must be structured, not ad-hoc

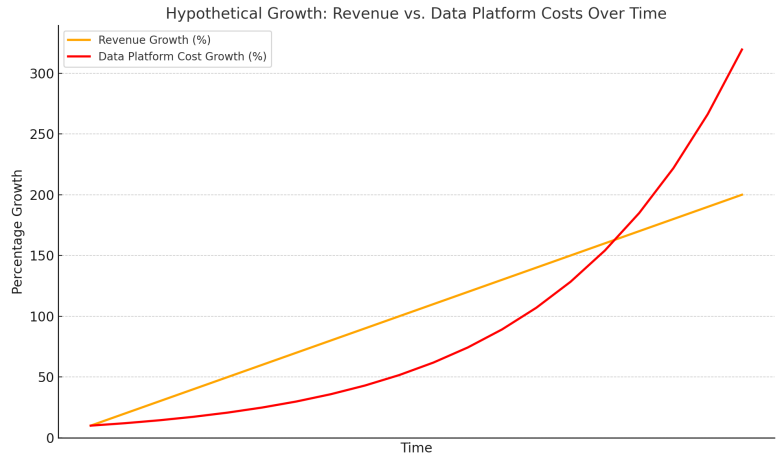
Prevention: reducing incident frequency

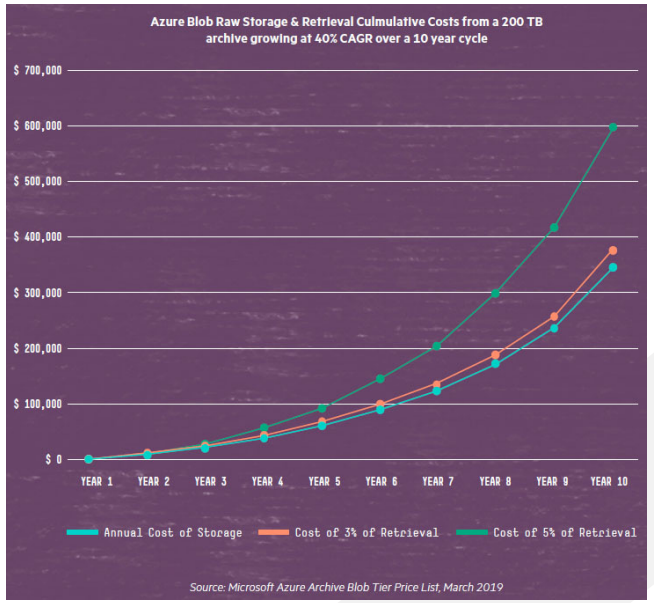
- Incidents are inevitable, silent data corruption is not
- Prevention comes from combining what we discussed in previous sections
  - Data quality: clear constraints and expectations
  - Testing: catch issues before deployment
  - Observability: detect problems early
- Additional good practices (already covered in previous classes)
  - Versioned data contracts
  - Safe deployment (testing > staging > production)
  - Backfills and reproducibility
  - Idempotent pipelines
- Anti-patterns (what to avoid!)
  - Manual fixes in production
  - No clear ownership
  - No postmortems
- In an organization, reliability is not a feature that you turn on or off (by using or not using a given technology), it is the result of systematic practices, of an installed cultured, of strong Governance

Performance and Cost

Cost is a system property

- Cost is often treated as a business concern, something to optimize later, after we prototyped a solution
- But, in data platforms, cost is a direct consequence of design / technical decisions
- Examples
  - Poor partitioning > more data scanned > higher cost
  - Bad joins > large shuffle > expensive queries
  - Recomputing data > duplicated compute cost
  - No materialization > repeated expensive queries
- The key idea to have in mind is that every design decision has a cost implication
- Therefore, cost must be considered during
  - Modeling
  - Processing
  - Serving





What drives cost in data processing

- At a high level, distributed query cost is driven by 4 main aspects
- Data read (I/O): how much data is scanned from storage. Influenced by
  - Partitioning
  - Predicate pushdown
  - Column projection
- Data movement (shuffle): data exchanged between workers. Triggered by
  - Joins
  - Aggregations
  - Window functions
- State and memory: intermediate data held during execution. Leads to
  - Memory pressure
  - Spills to disk
- Skew and imbalance
  - Uneven work distribution
  - One slow task delays the whole job
- Simply looking at costs from an aggregated point of view is not enough. We need to decompose the origin of costs, to then be able to tackle the problem

Cost per pipeline vs. cost per query

- Among other factors, cost depends on how often we compute something
- Query-on-read
  - Compute results at query time
  - Pros: flexibility, no precomputation
  - Cons: repeated expensive queries, unpredictable cost
- Materialization
  - Compute once, store results
  - Pros: predictable performance, lower cost per query
  - Cons: storage overhead, refresh cost
- Trade-off
  - Many users × same logic > materialize
  - Few users × evolving logic > query-on-read
- Often, we need to choose where we pay the cost (e.g. is storage cheaper than processing, for this specific query?)

Common cost anti-patterns

- Systems often become expensive due to avoidable design issues
- Recomputing everything
  - No incremental logic
  - No partition awareness
- Too many small files
  - High planning overhead
  - Inefficient scans
- Heavy queries in dashboards
  - Same logic executed repeatedly
- Unnecessary data movement
  - Joins without filtering
  - Wide tables without projection
- Lack of observability
  - No visibility into where cost comes from
- Asides from the inherent cost of a pipeline, cost problems are usually design problems

Optimization must be evidence-based

- Optimization should not be based on guesswork or trial-and-error without insight
- Instead, we use evidence
  - Execution plans
  - Runtime metrics
  - Data volume statistics
- Questions to ask
  - Where is time spent?
  - How much data is read?
  - Where does shuffle happen?
  - Are there skewed tasks?
- Examples
  - High input bytes > improve partitioning
  - High shuffle > reduce join size or pre-aggregate
  - Spills > reduce intermediate data
- Cost optimization is a repetitive process of measuring, understanding and only then optimizing
- This is also not a one-shot action, it should be a continuous effort, as pipelines then to become more expensive with time, as data accumulates

Scalable Technologies for Data Analysis

Quality, Tests, Observability and Costs

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026